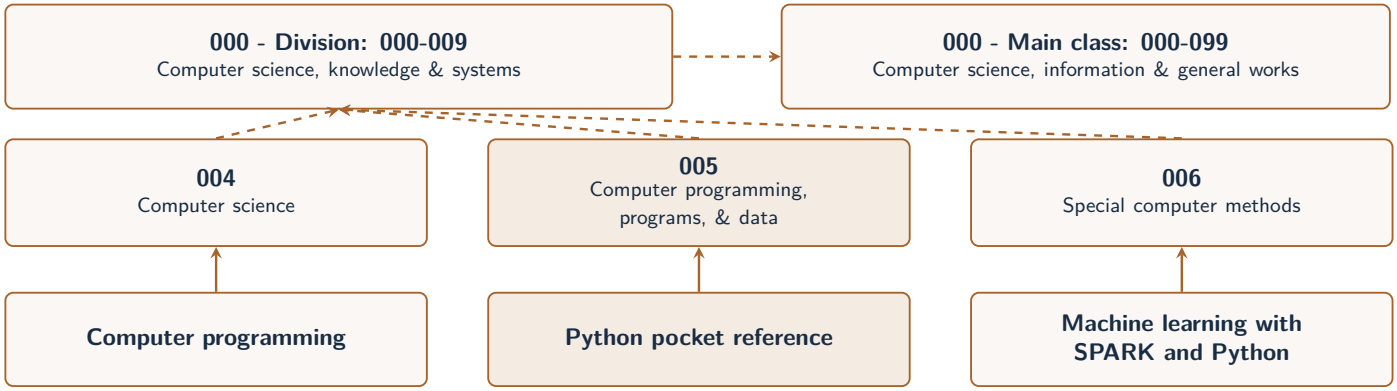
KOS Computing Structures

One computing theme. How does each system identify parent and child nodes?

01 DDC: document assignments and summary hierarchy

Label	Document title	Document description (excerpt)	D / A row
004	Computer programming	There are some unobvious dead lock problems caused by inappropriate acquiring and releasing resources among...	5384
005	Python pocket reference	Updated for both Python 3.4 and 2.7, this guide provides concise information on Python types and statements,...	6303
006	Machine learning with SPARK and Python	Machine Learning with Spark and Python Essential Techniques for Predictive Analytics, Second Edition...	6873

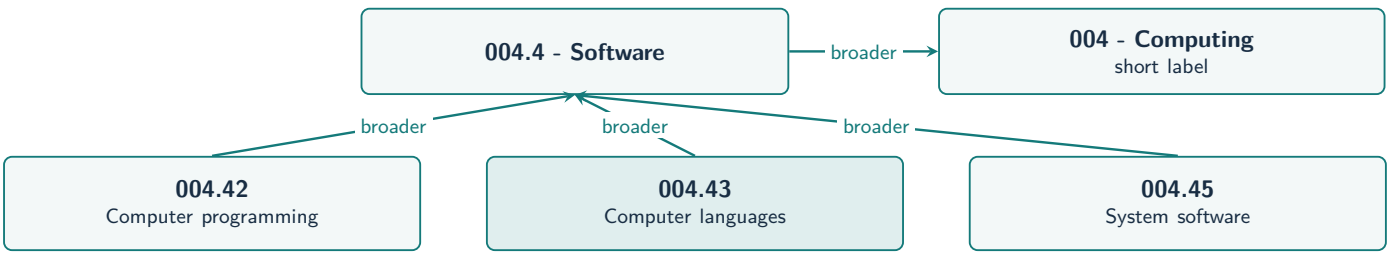


How to identify. The first, second and third digit positions indicate main class, division and section. Main class 000 and division 000 differ by level, not printed length. Beyond three digits, extending the same prefix usually narrows the subject: 004 -> 004.2. Confirm the parent using schedule headings and notes; exceptions exist. Dashed arrows point to broader classes; solid arrows assign documents, not child classes.

D/A: DDC_Prepared_Documents_001 / Assignments_001, rows above; Units 6-8. 004.2 illustrates notation, not a workbook row. OCLC summaries; 004.2 schedule

02 UDC: decimal subjects and broader concepts

Notation	Preferred label	Broader	Rows
004	Computer science and technology. Computing. Data processing	not shown	U657
004.4	Software	004	U678 / S3358
004.42	Computer programming. Computer programs	004.4	U683 / S3384
004.43	Computer languages	004.4	U684 / S3388
004.45	System software	004.4	U685 / S3392



How to identify. For simple numbers, extending a shared digit prefix normally narrows the subject: 004 -> 004.4 -> 004.43. Dots group digits for readability, not one dot per level. Confirm the direct parent using the schedule or skos:broader (subject = child, object = parent). Auxiliaries and combination symbols such as ":" must be interpreted first; blindly shortening compound numbers can give wrong parents.

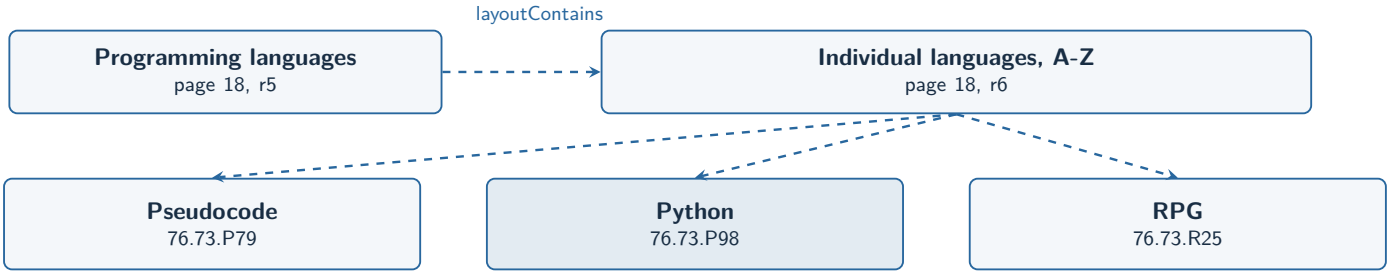
UDC_Summary_Prepared.xlsx; U = Units, S = Statements. No selected Python-specific node. UDC notation

Schedule Context and Subject Headings

LCC shows containment in a printed schedule; LCSH connects controlled subject headings

03 LCC: Python and its neighboring schedule entries

Row	Printed notation	Label	Units row
r5	(heading)	Programming languages	4652
r6	(heading)	Individual languages, A-Z – Continued	4653
r16	76.73.P79	Pseudocode	4663
r17	76.73.P98	Python	4664
r18	76.73.R25	RPG	4665

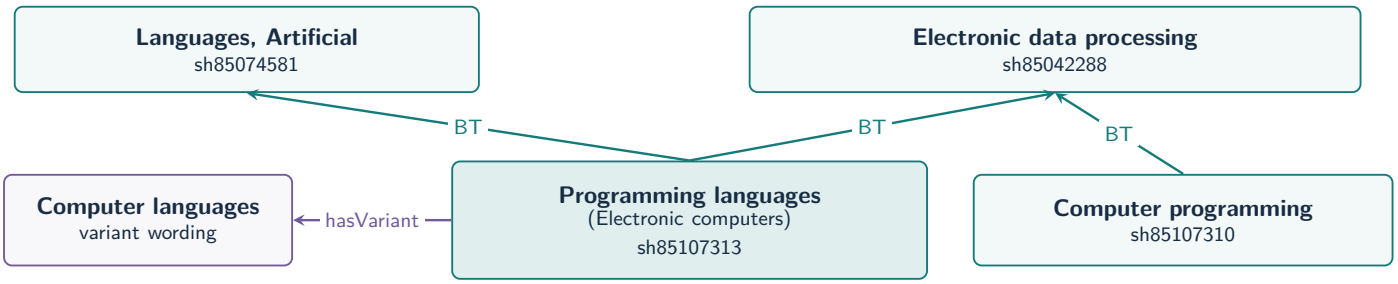


How to identify. Locate a classmark within the schedule's enclosing number ranges, then follow headings and indentation: more-indented entries belong under the enclosing heading; equally indented entries under it are peers. More digits or a longer suffix do not automatically mean a deeper level. Read QA and continuation headings as context. Our derived layoutContains arrows run parent -> child; .P98 identifies the Python entry, not an extra level.

LCC_Prepared.xlsx, Units rows above; Statements 4264, 4274-4276. Pseudocode is outside the 10,000-row subset. LC schedule guide

04 LCSH: controlled headings and alternative wording

Subject heading	Relation	Object heading	Row
Programming languages (Electronic computers)	hasBroaderAuthority	Languages, Artificial	11986
Programming languages (Electronic computers)	hasBroaderAuthority	Electronic data processing	11724
Computer programming	hasBroaderAuthority	Electronic data processing	11650
Programming languages (Electronic computers)	hasVariant	Computer languages	11787



How to identify. Check the relation field: hasBroaderAuthority means subject = child and object = broader parent; hasNarrowerAuthority reverses those roles. One subject linked to several broader objects has multiple parents. The sh identifier digits and label wording do not encode depth. hasVariant is alternative wording, not parentage. These links organize subject access, not formal class membership.

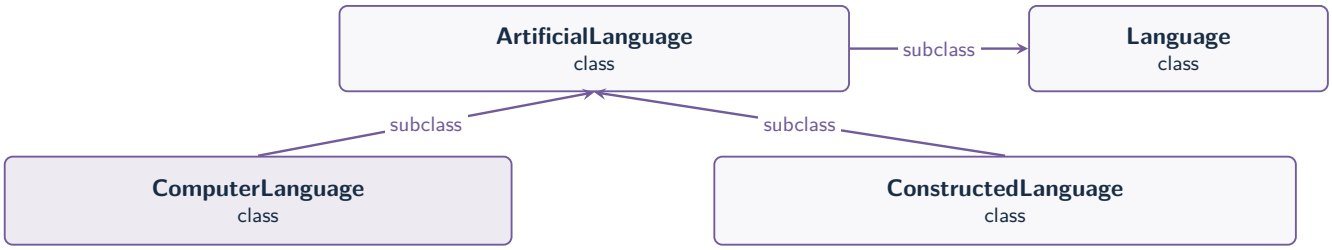
LCSH_Statements_Part007, Statements rows above; variant text: LCSH_Texts_Part002, Texts 12504. LC MADS/RDF

Formal Classes and Wikidata Statements

Subclass arrows run from narrower class to superclass; instance membership is a different relation

05 SUMO: formal classes and an inheritance rule

Class	Relation	Superclass	Logic row
ComputerLanguage	subclass	ArtificialLanguage	457
ConstructedLanguage	subclass	ArtificialLanguage	466
ArtificialLanguage	subclass	Language	455



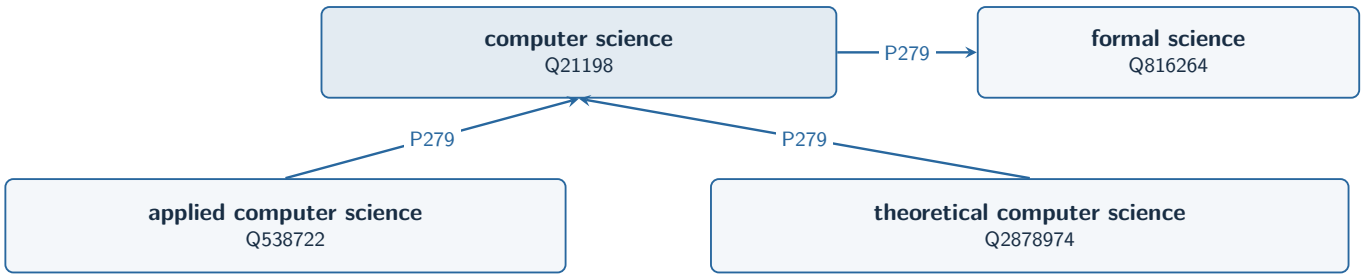
(=> (and (subclass ?X ?Y) (instance ?Z ?X)) (instance ?Z ?Y))

How to identify. Read the predicate and argument order: in (subclass C P), C is the child class and P its parent. Repeated C with different P values gives multiple parents. (instance x C) instead assigns a member to a class. Symbol names do not encode hierarchy. The displayed rule makes every member of a child class a member of its parent; no named instance is asserted.

SUMO_Prepared_Native_Logic.xlsx, Native_Logic rows above; rule at row 26 (line breaks normalized). SUMO source

06 Wikidata: items connected by named properties

Subject item label	Property	Object item label	Row
applied computer science	P279 (subclass of)	computer science	72803
theoretical computer science	P279 (subclass of)	computer science	51952
computer science	P279 (subclass of)	formal science	8710



How to identify. Find P279 statements: the subject item is the child class and the value item is its parent. Follow another P279 statement from that parent to find the next ancestor. Several P279 values can give one child several parents. Q numbers and label wording do not encode depth or parentage. P31 instead identifies instance membership; it is not a subclass link.

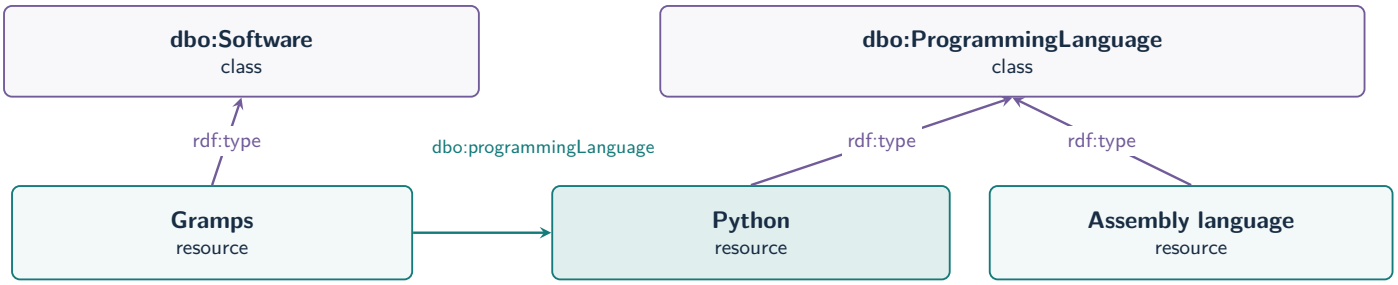
Wikidata_Prepared_Statements_001.xlsx, Statements rows above. This sample is at discipline level. Wikidata statements

Typed Resources and Data Coverage

Sharing a type is different from sharing a superclass; the relation label decides how to read the graph

07 DBpedia: resource types and facts between resources

Subject resource	Relation	Object resource / class	Row
Gramps (software)	dbo:programmingLanguage	Python (programming language)	881
Gramps (software)	rdf:type	software	3103
Python (programming language)	rdf:type	programming language	3583
Assembly language	rdf:type	programming language	2744



How to identify. To identify class parents, look for rdfs:subClassOf in the ontology: subject class = child, object class = parent. Keep multiple parents when recorded. This example contains rdf:type class membership, not subclass edges, so it cannot establish class parents. URI spelling cannot supply missing hierarchy. programmingLanguage is a software fact, not a parent-child relation.

DBpedia_Prepared.xlsx, Statements rows above. dbo: = http://dbpedia.org/ontology/. DBpedia ontology

What evidence identifies a parent?

Systems	What to inspect	Avoid this shortcut
DDC / simple UDC	Shared number prefix and meaningful positions; confirm with schedule or broader link.	Do not equate printed length with level or blindly truncate.
LCC	Enclosing class-number ranges, headings, indentation and schedule instructions.	More digits or a longer suffix need not mean a child.
LCSH / FAST	Recorded broader relation: subject = child, object = parent; narrower is reversed.	Identifiers and similar labels do not encode parentage.
SUMO / Wikidata / DBpedia / YAGO	subclass argument order, P279, or rdfs:subClassOf: child -> parent.	instance, P31 and rdf:type are memberships, not subclass links.
BabelNet research subset	hypernym / subclass_of: child -> parent; hyponym reverses direction.	Synset IDs and same-synset word forms do not encode a hierarchy.

The shared computing theme is not an exact cross-system mapping. The first seven examples use retained prepared workbooks, including some neighbors outside the 10,000-row subsets; FAST and YAGO use their 10,000-row workbooks; BabelNet uses a research subset. Row numbers include the header. A missing edge here does not mean the full KOS lacks it.

Ten KOS are included: FAST and YAGO 4.6 on page 5; BabelNet research subset on page 6. WordNet remains deferred.

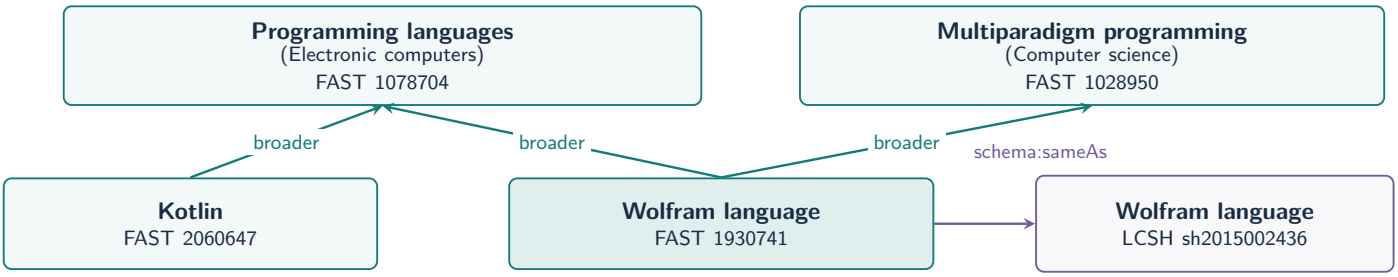
Background for interpreting broader, type and subclass relations: W3C RDF Schema

FAST Headings and YAGO Classes

The original computing examples at two levels: subject headings and classes of code-related things

08 FAST: faceted headings and a cross-vocabulary link

FAST subject heading	Relation	Object heading	Row
Kotlin (Computer program language)	skos:broader	Programming languages (Electronic computers)	7588
Wolfram language (Computer program language)	skos:broader	Programming languages (Electronic computers)	7917
Wolfram language (Computer program language)	skos:broader	Multiparadigm programming (Computer science)	7916
Wolfram language (Computer program language)	schema:sameAs	[LCSH] Wolfram language (Computer program language)	6672

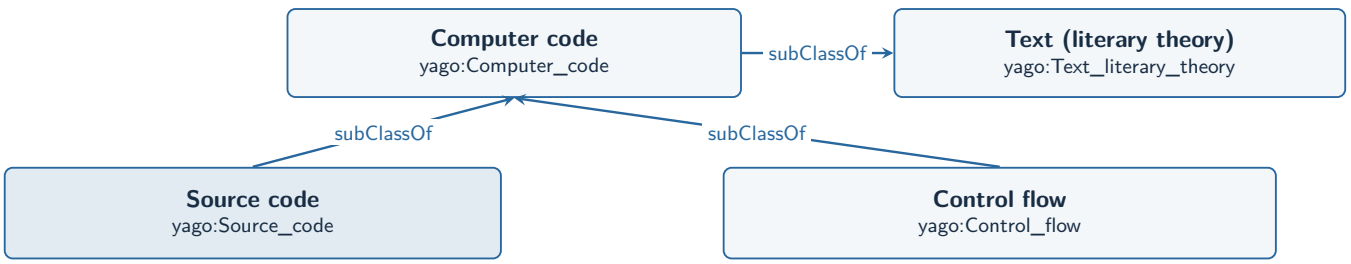


How to identify. For skos:broader, the subject is the child heading and the object its broader parent; skos:narrower reverses this direction. Several broader objects for the same subject URI mean multiple parents, as for Wolfram here. FAST number length and heading wording do not encode levels. schema:sameAs connects records across vocabularies, not parent and child; broader organizes subject access, not class membership.

FAST_10000.xlsx, Statements. No Python record here; both broader targets have no selected outgoing records. OCLC FAST

09 YAGO 4.6: classes in a knowledge-graph taxonomy

Subclass label	Relation	Superclass label	Row
Source code	rdfs:subClassOf	Computer code	991
Control flow	rdfs:subClassOf	Computer code	669
Computer code	rdfs:subClassOf	Text (literary theory)	659



How to identify. Filter taxonomy rows for rdfs:subClassOf: the subject class is the child and the object class its parent. Follow successive rows to find ancestors and retain all recorded parents. An underscore or a longer YAGO name does not signal another level. rdf:type records membership, not parentage. The shown Control flow placement and Text label are retained source assertions.

YAGO_4_6_10000.xlsx, Hierarchy rows above. yago: = http://yago-knowledge.org/resource/. YAGO 4.6 model

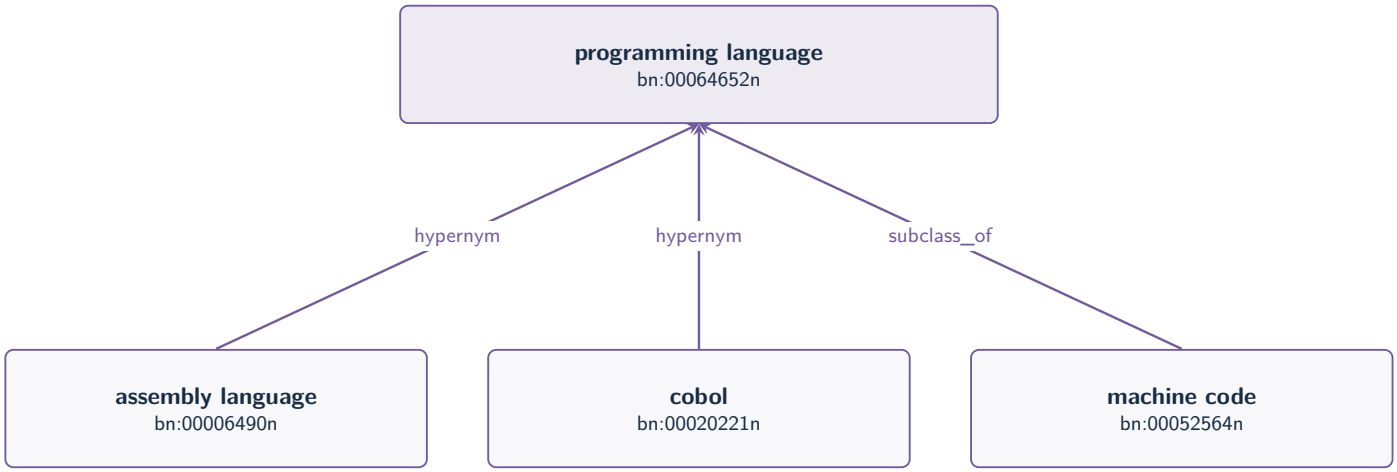
BabelNet: Word Senses and Broader Concepts

A research subset: programming-language meanings connected by their recorded relations

10 BabelNet: a shared broader concept

A node is a synset: one specific meaning, identified by a BabelNet ID. The three selected meanings share programming language as a broader concept.

Source meaning / synset	Relation	Target meaning / synset	Row
assembly language (bn:00006490n)	hypernym	programming language (bn:00064652n)	2
cobol (bn:00020221n)	hypernym	programming language (bn:00064652n)	3
machine code (bn:00052564n)	subclass_of	programming language (bn:00064652n)	4



Arrows preserve the source relation types and point to the parent. The synset: prefix is omitted for readability.

How to identify. Read the native relation: hypernym / subclass_of means source = child and target = parent; hyponym reverses those roles. Follow recorded parent links for ancestors and retain multiple parents. The numeric bn: ID does not encode depth. Word forms and translations within one synset name the same meaning; they are not child nodes. Instance links express membership, not subclassing.

This 10,000-row selection is from a historical research subset, not the complete or current BabelNet release. English display labels come from the MSGI snapshot matched by synset ID; they are not claimed as official preferred labels.

Assembly language also appears in the DBpedia example; the shared wording is a comparison aid, not an asserted exact mapping.

Source: BabelNet_10000.xlsx, Statements rows above; relations: THUNLP BabelNet-Sememe-Prediction

English words and definitions matched by BabelNet ID: THUNLP MSGI

The sample retains hypernym and subclass_of as distinct source predicates; it does not turn every lexical link into a formal subclass axiom.